

A PAPER · OCTOBER 2026

Width, font size, tracking: *who really does the fitting?*

[Slides ↗](#) · [Data ↗](#) · [Paper PDF ↓](#) · [Transcript ↗](#) · [Demo ↗](#)

CONTENTS

- 01 Summary
- 02 The idea, and who had it first
- 03 Who ships a width axis
- 04 How far the axis reaches
- 05 In Roboto Flex, the reach changes with size
- 06 Families with a long width range
- 07 What fitWidth does with it
- 08 The cost of tracking
- 09 What CSS is doing
- 10 The order we recommend
- 11 Limits
- 12 Objections, answered
- 13 Open questions
- 14 Method
- 15 Sources

Quinn Keaveney, Overpunch · October 2026

A variable font's width axis looks like the right tool for fitting a headline to its box. This paper counts how many open fonts have one, measures how far it moves a headline in 21 of them, and tests our own fitting library on those fonts. In the fonts we measured, the axis is a modest adjustment, and something else does most of the fitting.

Disclosure: the author makes fitWidth and other type tools for the web at Overpunch, which has built websites for type foundries for over fifteen years and is building Typetin, a storefront platform for independent foundries (in development, not yet launched). The measurements use open tools (HarfBuzz, fontTools, Chromium) and open fonts. Our first recommendation doesn't need our library, and the order we argue for is not yet in its npm release.

Summary

Of 1,950 families on Google Fonts, 97 have a width axis (`wdth`). In 49 of those the axis stops at 100, so it can narrow a headline but can't widen it. Thirty-seven of the 49 are Noto families, one design system released for many scripts; without Noto it is 12 of 60.

We measured 21 of the 97 in Chromium, over `wdth` 75–125, which is fitWidth's default search range. A headline's *natural width* here means its width at `wdth` 100 at the size set. **Half of the 21 couldn't take a headline below 80% of its natural width, and half couldn't take it past 113%.** Those are two separate medians; no single font has that range. Six can't widen at all; the 15 that can reach a median of 119%. Families with a long width range go much further.

We then asked fitWidth to fit one headline in each of the 21 fonts to 16 target widths, from half to double its natural width. The axis alone fitted **75 of 336**. Without the 21 targets where nothing has to move, it is 54 of 315.

So when a box is further away than the axis can reach, something else is doing the fitting. In fitWidth's released default that is letter-spacing (tracking). We now think it should be font size, with tracking held to a small cap.

What is released. fitWidth 1.1.0 is on npm. The font-size step (`size`), the returned result and an option described below (`trimTrailingSpace`) are in the repository and run on fitwidth.com; they are not on npm yet, and `size` is off by default. Wherever a number below comes from the unreleased build, it says so.

The idea, and who had it first

The OpenType specification suggests it. The registration of the width axis says:

Applications may choose to select a width variant in a variable font automatically in order to fit a span of text into a target width.

The same page is careful about scale: applications "may choose to make small, automated 'wdth' adjustments", the change in value "may be used as a first

approximation", and the application "will likely need to refine the adjustment over multiple attempts".

Laurence Penney built it. His [fit-to-width.js](#) (2018; its README says the method was "first presented" on the Axis-Praxis blog in November 2016) "automatically adjusts a variable font's width (`width`) axis, as well as adjustments of letter-spacing and word-spacing", by binary search, as a sequence of operations. `fitWidth`'s default is the same idea. What this paper adds is the count of fonts, the measured reach, and font size as the step after the axis.

The request reached CSS in 2018. The issue asked for text that always fits its parent, by any method. The same day, Sergey Malkin commented: "Changing font-stretch, especially using variable fonts, is another way to fit text into parent." Seven years later Peter Constable wrote in that thread: "Since OpenType Font Variations was introduced in 2016, the idea of manipulating the 'width' axis of variable fonts to fit text to a target width has often been raised." He was arguing that a feature which only changes size should be named for what it does.

Four popular fit-text libraries (`fitty`, `textFit`, `FitText.js`, `BigText`) fit by font size; `BigText` adds word-spacing. None of their READMEs mentions `width`, `font-stretch` or variable fonts. We found no published measurement of how far the axis moves a line across a set of fonts.

Who ships a width axis

We read the Google Fonts catalogue metadata on 7 October 2026.

Families

All families	1,950
Variable	561
With a <code>width</code> axis	97
of which Noto / Anek (one design per script)	37 / 10
Axis stops at 100 (can't widen)	49, all 37 Noto among them
Axis starts at 100 (can't narrow)	8
Covers all of 75–125	37
Exactly 75–125	24

The median axis spans 37.5 units. The most common range is 62.5–100 (the Noto families); the next is 75–125. Every family and its range is on the [data page](#).

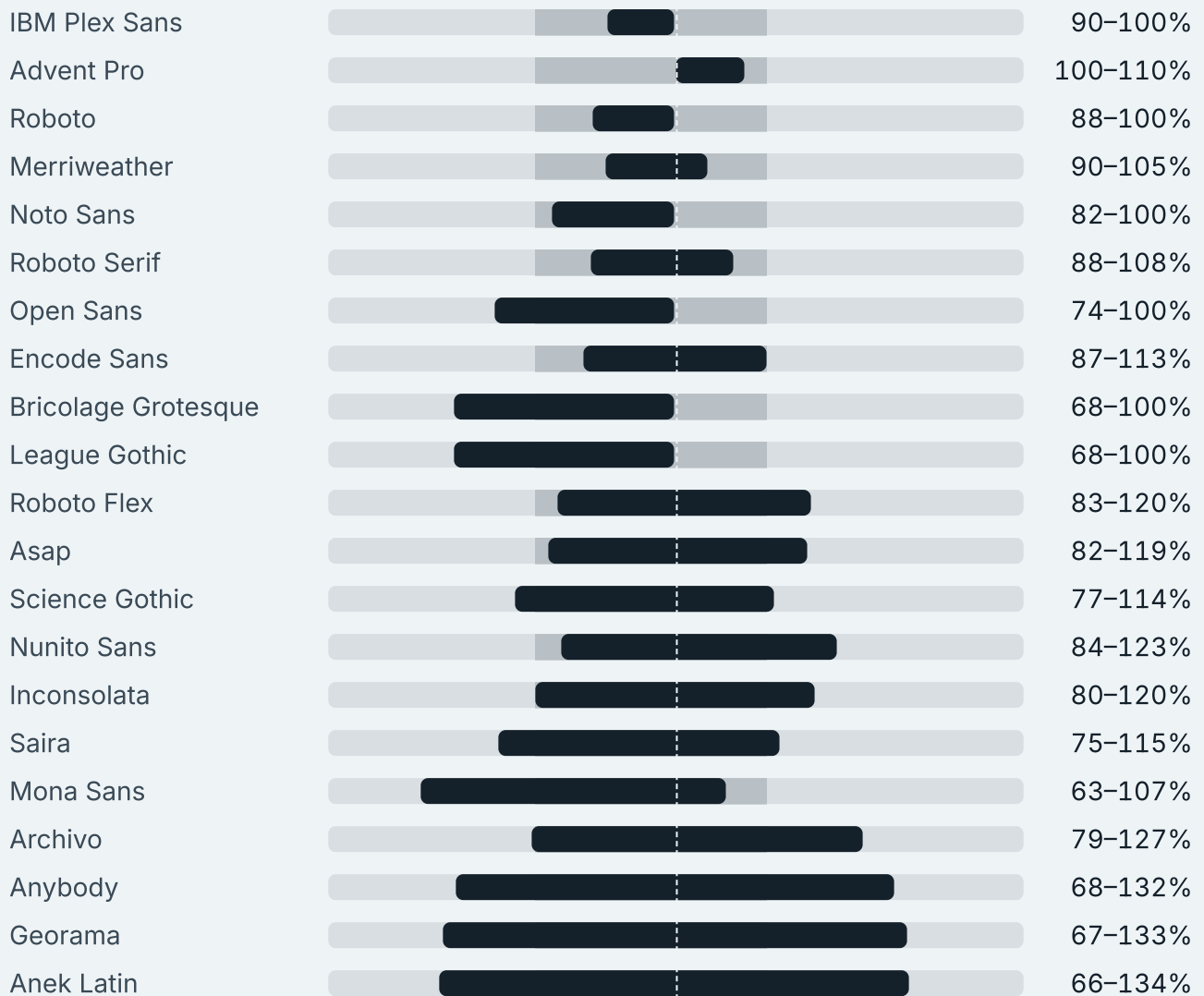
Two cautions. This is one catalogue of free fonts, and retail families may be drawn with longer ranges; we didn't measure any. And the count is of families: Noto and Anek repeat one design across scripts.

How far the axis reaches

A `width` value reads like a percentage, and the specification says values "can be interpreted as a percentage of whatever the font designer considers "normal width"". In practice it is the designer's label for a design. The spec says as much in the next breath: the value "may be used as a first approximation".

We took 21 of the 97 families and measured five headline strings in each at `width` 75, 100 and 125 (or as close as the font allows), at 72 px and weight 400. *Reach* is the line's advance width at 75 and at 125, divided by its advance width at 100. Advance width includes the spacing the designer gave each letter at that width.

How far width 75–125 moves a headline: half stop by 80%, half by 113%.



Scale 50% to 150% of the width at width 100; the dashed line is 100% and the shaded band runs between the two medians. Mean of five strings at 72 px, weight 400, Chromium 149, fonts from google/fonts at commit 7085eb8, 7 October 2026.

- Half of the 21 stop at or above 80.0% at the narrow end; half stop at or below 113.1% at the wide end.
- 6 can't widen. 8 span less than 30 percentage points; 5 span less than 20.
- The axis is not linear, and often not symmetric about 100. At the ends of each font's own range, the measured width differed from the `width` number by a median of 9 percentage points. IBM Plex Sans at 75 is 90% wide. Inconsolata, a monospace, matches at the ends of its range (50 and 200) and not between: at 75 it is 80% wide.
- Reach is about the line, not the letters. Mona Sans at 125 is 107% wide, but its "n" is 19% wider in ink: the expanded design is spaced 25% tighter. That is the designer's

spacing decision, and it is why the line grows less than the letters.

- Reach varies by a few percentage points with the letters in the line, and it shrinks as weight goes up: at weight 700 the medians are 83% and 113% (HarfBuzz, 20 families).
- HarfBuzz and Chromium agreed within 0.10% across all 315 widths compared (21 fonts × 5 strings × 3 values).

The 21 were picked by hand to cover well-known families, and the sample is kinder to the axis than the whole set: 6 of these 21 can't widen, against 49 of the 97.

In Roboto Flex, the reach changes with size

Five of the 21 have an optical-size axis, an axis that changes the drawing for small or large sizes and that browsers set from the font size. In Roboto Flex the width axis is drawn short at small sizes and long at large ones.

Roboto Flex: the same axis reaches further as the type gets bigger.



Width at wdth 75 and at wdth 125 as a share of the width at wdth 100, at each font size, with optical size following the size automatically. Scale 70% to 130%. Mean of five strings, Chromium 149; HarfBuzz gives the same figures at the matching opsz.

At 144 px, `width 75` takes 20.3% off the width. At 36 px, a phone headline, it takes 10.9% off. At 14 px, 3.6%. Small type can't be squeezed and stay readable, so the designers didn't draw it squeezed.

Font size isn't linear in this font either: going from 36 to 72 px multiplied the width by 1.84, not 2. Roman Komarov saw the same in Fraunces when fitting with CSS: "the optical adjustment of the larger sizes makes the glyphs narrower, making the proportional increase not fill the lines fully."

Roboto Flex is the extreme case. The effect is smaller in Bricolage Grotesque and Roboto Serif, runs slightly the other way in Merriweather, and is absent in Nunito Sans and in the 16 families with no optical-size axis. Roboto Flex also has a parametric axis, `XTRA`, that changes counter width and leaves stems alone; it reaches 77–127% at 72 px. Very few families have one.

Families with a long width range

Some families go much further. Over 75–125, Anybody reaches 68–132%, Georama 67–133% and Anek Latin 66–134%; Mona Sans narrows to 63%. Over its full 50–150, Anybody reaches 37–163% (HarfBuzz, weight 400; 46–154% at weight 700).

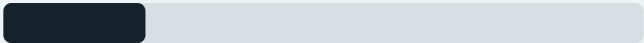
David Jonathan Ross's Fit is the clearest case outside the sample. He describes it as "designed with one thing in mind: filling up space with maximum impact", and says: "I drew Fit as a variable font with an expansive range of widths". We didn't measure Fit (it's a retail font), so we only quote him.

If fitting by width is what your design needs, choosing a family like these does more than a script can: a script can't add range the designer didn't draw.

What fitWidth does with it

fitWidth searches the width axis first, at the font size you set, then closes what is left with letter-spacing, up to $\pm 0.3\text{em}$. We ran it on the same 21 fonts in Chromium: one headline ("Headline fitting", 72 px) and 16 targets from 0.5× to 2× its natural width, in steps of 0.1×.

21 fonts × 16 target widths: the axis alone fitted 75 of 336.

Width axis only		75
Tracking only, ±0.3em		267
Axis, then tracking (default)		294
Axis, then size, then ±0.05em		336

"Headline fitting" at 72 px in each of the 21 fonts, targets from 0.5× to 2× natural width in steps of 0.1×. fitWidth built from its repository, Chromium 149; the first three rows write the same styles as npm 1.1.0, and the last needs the unreleased size option. A fit is never wider than the target and at most 0.5 px narrower.

Strategy	Fits, of 336	Without the 1.0× targets, of 315
Width axis only	75	54
Tracking only, ±0.3em	267	246
Axis, then tracking (the default)	294	273
Axis, then font size 0.5–2×, then tracking ±0.05em (not released)	336	315

The first three rows are also what published 1.1.0 does: across those 1,008 fits the repository build and 1.1.0 wrote identical styles. The last row needs the unreleased `size` option.

Read the table with care. The counts describe this grid of targets, not headlines in general; nothing here says what boxes real layouts ask for. The targets run from 0.5× to 2×, and the size option also runs from 0.5× to 2×, so the last row fits everything by construction. A library that only scales the font, with no limit, would fill every one of these boxes too. What the axis buys in the last row is the 75 cases (54 of them non-trivial) where the size you set didn't have to change.

And the default's 294 fits include many nobody would ship. 171 of them needed more than 0.05em of letter-spacing; 107 needed more than 0.12em, which is the most Matthew Butterick suggests even for capitals. The test headline is lowercase.

Every status was checked against the rendered element afterwards (336 targets × 4 strategies = 1,344 checks, no disagreements).

The cost of tracking

Letter-spacing reaches far. A `width` step changes spacing too, but with sidebearings and kerning the designer drew for that width. Letter-spacing overrides them with one number for every pair. Butterick's default for lowercase is none:

Lowercase letters don't ordinarily need letterspacing.

He adds that typographers "will often remove letterspacing from lowercase text used at larger sizes (e.g., headlines)". Capitals are different: "you always add 5–12% extra letterspacing to text in all caps or small caps."

Three costs showed up in testing.

Ligatures switch off. The CSS Text specification says browsers "should not apply optional ligatures" when letter-spacing is not zero, and Chromium doesn't. In Advent Pro, "Headline fitting" at 72 px is 402.3 px wide at `width` 125. Add any spacing and its "tt" and "fi" ligatures split (8.5 px and 1.4 px), and the line jumps to 414.0 px. No tracking value lands on a target between the two. In 15 of the 21 fonts this headline's jump is about half a pixel or less; in Georama and Advent Pro it is over 10 px.

The last letter misses the edge. Chromium adds letter-spacing after the last letter as well as between letters, and `fitWidth` counts that trailing space as part of the width. So a tracked fit ends with its last letter short of the edge by the tracking amount, or past it when the tracking is negative. In the 219 default fits above that added tracking, the last letter ended between 21.9 px short and 12.7 px past the target, a median of 8.5 px away. Penney's README noted the trailing space in 2018. We tried cancelling it by default and a review of that build showed it breaks ordinary layouts (a heading with `width: 100%` wraps), so it is an opt-in, `trimTrailingSpace`, for elements that size themselves to their text. With it, 211 of those 219 still fit and end 0 to 0.5 px inside the target; the other 8 reach the tracking cap sooner. It is not released.

Negative tracking collides. A narrow box makes the default fit with negative tracking. In the demo's opening font, by about -0.24em the letters overlap. The width matches and the word can't be read.

What CSS is doing

CSS is adding a fitting feature of its own, and it works by size. The 2018 issue was closed on 28 April 2026, with Tab Atkins writing: "I'm going to close this issue as the core feature is done." The feature is `text-fit`, in the CSS Text Level 5 editor's draft, which says it "scales the font size".

```
/* Scale each line up to fill its container, to at most double. */  
h1 { text-fit: grow per-line-all 200%; }
```

The width axis isn't in that draft section. The Chrome team's explainer, which calls itself "an early design sketch", raises "`font-weight` or `font-width`" as possible fitting methods and answers, in a note in its source: "They work well only with specific fonts, and they don't offer the flexibility to fit text to any width. So we don't apply them in the initial proposal." Our measurements agree. Komarov has asked that the design keep room for other methods, variable axes among them.

We tested Chromium 149: `text-fit` is not supported by default, and with experimental web platform features turned on it took a 334.5 px headline to 600 px. We didn't test Safari or Firefox. The same explainer says the feature "has not been approved to ship in Chrome".

Until it ships, a fluid font size gets close with no script:

```
.card { container-type: inline-size; }  
.card h1 { font-size: clamp(2rem, 11cqi, 6rem); }
```

That scales with the container, not with the length of the text, so it doesn't land on the edge. Often that is all a design needs.

The order we recommend

1. **Scale the text** if filling the box is the whole goal. Use `text-fit` if it ships, and fluid sizes or a size-fitting script until then. No width axis is needed.

2. **Pick a family with a long width range** if the look you want is letters that change shape to fill the line.
3. **Use the width axis for the last 10–20%** of a single line, when the size is fixed for a reason. Within a set of headlines seen together, keep to about one width class.
4. **Let font size take over when the axis runs out**, within limits you set.
5. **Keep letter-spacing small**: none for lowercase if you can. Capitals can take more, and take it better than lowercase does.

Steps 3 to 5 are what `fitWidth`'s unreleased `size` option does:

```
applyFitWidth(el, { size: true })  
// axis first; font size 0.5× to 2× only if the axis can't reach;  
// then at most ±0.05em of letter-spacing
```

It also returns what each step did (the axis value, the font size, the tracking, the width each contributed and whether the text fits), which is where the figures under each row of the [demo](#) come from.

Limits

- **Font size costs height.** A fitted headline that grows from 64 to 91 px is a taller line, the layout below it moves, and it may read as a different level of the hierarchy. That's why `size` is opt-in and capped.
- **It can fail where the default succeeds.** Font size stops at 0.5×, so in a very narrow box the text overflows; the default would squeeze it in with -0.3em of tracking, illegibly.
- **Width changes weight and proportion.** In Anybody, Archivo, Mona Sans and Anek Latin the stem of an "l" is about 25% thicker at `wdth` 125 than at 75. Headlines fitted to very different widths, seen side by side, can read as different fonts.
- **Scaled text and accessibility.** Text sized to its container may not grow when a reader zooms, which can fail WCAG 1.4.4, Resize Text. Komarov has proposed that CSS default to "a default limit equal to 200% of the original font-size"; the Chrome team has said it is not sure that limit works well. `size: true` stops at 2×, and we have not tested fitted headlines under zoom.

- **The search range is a choice.** 75–125 is fitWidth's default, not a property of fonts. Roboto Flex reaches 83–120% over it and 49–140% over its full 25–151. The library doesn't read a font's own range, so you pass it. At the ends of a full range the letters look like a different typeface, so use them by choice.
- **The search assumes width rises with `width`.** In Mona Sans, "illicit" gets narrower above 100 (150.4 px at 100, 146.8 px at 125, at 72 px), and the axis step ends on the wrong side.
- **Nothing forces a fit.** Past its ranges the library stops and leaves the text short or overflowing. An overflow prints a console warning; the unreleased result object reports both.
- **Latin, one line.** Letter-spacing breaks joined scripts, and we tested Latin only. fitWidth fits a single line, needs the font loaded first, and measures a hidden copy about 20 times per step.
- **"Fits" is about the advance box.** The ink sits a few pixels inside it, depending on the first and last letters' sidebearings.

Objections, answered

"Type designers know a width range is narrow." They do. What we add is the count, the measured reach, the size dependence in one family, and font size as the next step. A developer who reads `width` 75 as 75% will measure something else: 90% in IBM Plex Sans.

"Search the font's whole range, then." For Roboto Flex or Anybody it helps a lot. For the 49 families that stop at 100 there is nothing more to search.

"Google Fonts isn't the fonts I license." This is a measurement of one open catalogue. We'd like the same numbers for retail families.

"Your default tracks to 0.3em." It has since 1.0, and changing a default changes existing sites, so the new order is opt-in. Whether it should become the default in a major version is an open question.

"Size fills every box. What does the axis add?" No fits: only the cases where the size you chose can stay. If the size doesn't matter to you, skip the axis.

"Changing size breaks my hierarchy." Then stay with the axis and accept that some boxes won't be filled. The result tells you by how much.

"Just use text-fit." If scaling is what you want, and it ships, yes. That's recommendation 1.

Open questions

- How do readers judge a headline fitted by width, by size and by tracking? We found no study, and ran none.
- What do retail width families reach?
- What target widths do real layouts ask for?
- Should fitWidth's default change to the size order in a major version? Should tracking be capped differently for capitals and lowercase, and should word-spacing come before letter-spacing?
- Do Safari and Firefox clamp, shape and space the same way? Every browser number here is Chromium.
- Will `text-fit` grow a way to use a font's axes?

Method

- **Census.** fonts.google.com/metadata/fonts, fetched 7 October 2026, 10:44 UTC. A family counts as having a width axis when its metadata lists a `width` axis.
- **Fonts.** 21 upright variable TTFs from [google/fonts](https://github.com/google/fonts) at commit `7085eb8` (5 October 2026): Advent Pro, Anek Latin, Anybody, Archivo, Asap, Bricolage Grotesque, Encode Sans, Georama, IBM Plex Sans, Inconsolata, League Gothic, Merriweather, Mona Sans, Noto Sans, Nunito Sans, Open Sans, Roboto, Roboto Flex, Roboto Serif, Saira, Science Gothic.
- **Reach.** Five strings ("THE WIDTH AXIS", "Headline fitting", "Hamburgefonstiv", "Breaking News: Markets Rally", "MINIMUM") at `width` 75, 100 and 125, each clamped to the font's range; weight 400; 72 px with optical size automatic. Reach is the mean over the five strings of advance width ÷ advance width at `width` 100. `width` 100 is what

CSS asks for by default; it is not the default instance in Encode Sans or Georama. Measured with HarfBuzz (uharfbuzz 0.56.1) and in Chromium 149.0.7827.55 (`getBoundingClientRect`), which agreed within 0.10%. Full-range reaches, weight 700 and the glyph measurements are HarfBuzz and fontTools only.

- **Library.** `fitWidth` built from its repository at commit [4aaaf917](#) (the 1.2.0 candidate), and published 1.1.0 from npm (checksum verified), both loaded in Chromium 149. Fits: "Headline fitting", 72 px, targets = natural width × 0.5, 0.6 ... 2.0. A fit is a measured width no wider than the target and at most 0.5 px narrower.
- **Not measured.** Safari, Firefox, retail fonts, italics, non-Latin text, zoom, and readers.

The tables are on the [data page](#). The scripts and their outputs are available on request.

Sources

- OpenType specification, `width` axis: [learn.microsoft.com](https://learn.microsoft.com/en-us/typography/opentype/spec/width)
- Laurence Penney, `fit-to-width.js`, 2018: github.com/Lorp/fit-to-width
- CSS Working Group issue 2528, "Feature for making text always fit the width of its parent", opened 11 April 2018, closed 28 April 2026: github.com/w3c/csswg-drafts. Malkin, [11 April 2018](#); Constable, [2 May 2025](#); Komarov, [1 October 2025](#); Atkins, [28 April 2026](#)
- CSS Text Module Level 5, editor's draft source, `text-fit`: github.com/w3c/csswg-drafts (the commit of 4 June 2026)
- CSS Text Module Level 3, editor's draft, letter-spacing and ligatures: drafts.csswg.org/css-text-3
- Blink Layout Team, "CSS fit-width text Explainer", README at the commit of 19 March 2026: github.com/explainers-by-googlers/css-fit-text. The sentence about `font-width` is inside an HTML comment in the file's source and doesn't show on the rendered page.
- Roman Komarov, "Text Fitting: Default scaling limit", CSSWG issue 12886, 1 October 2025: github.com/w3c/csswg-drafts
- Roman Komarov, "Fit-to-Width Text", July 2024: kizu.dev/fit-to-width
- Matthew Butterick, *Practical Typography*, "Letterspacing": practicaltypography.com
- David Jonathan Ross, Fit: djr.com/fit

- Google Fonts catalogue metadata: fonts.google.com/metadata/fonts; font files: github.com/google/fonts
- Library READMEs, read 7 October 2026: [fitty](#), [textFit](#), [FitText.js](#), [BigText](#)
- WCAG 2.1, Success Criterion 1.4.4 Resize Text: [w3.org](https://www.w3.org)